

# Raport Końcowy: Projekt 8

## Mini Lakehouse for Weather Data

Adam Błażejowski, 198344  
Mateusz Kuczerowski, 197900

29 czerwca 2026

### 1 Cel i Zakres Projektu

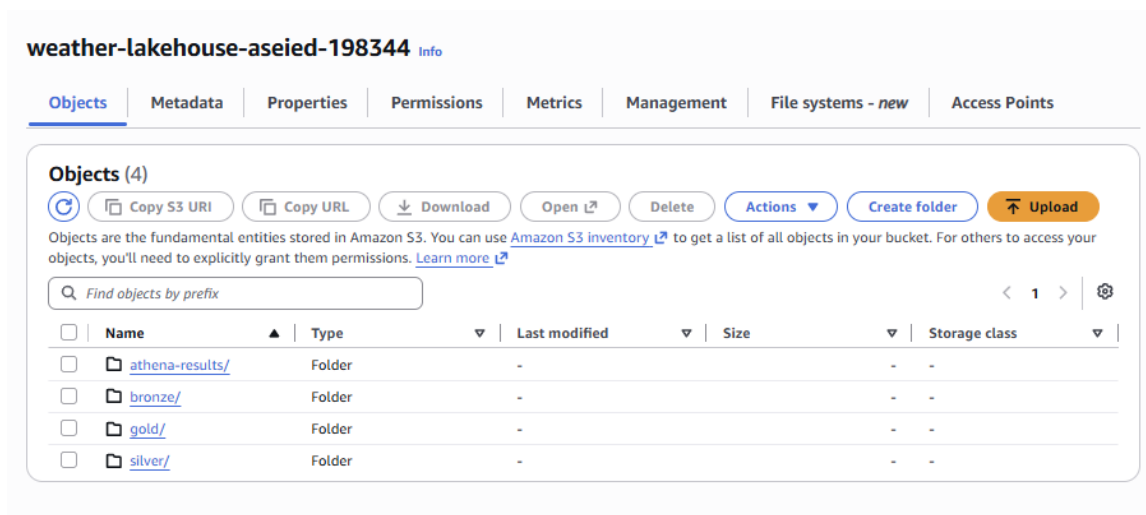
W ramach tego projektu zaimplementowany został kompletny potok danych (data pipeline), który realizuje architekturę klasy Data Lakehouse w środowisku chmury AWS. Celem systemu jest automatyczne pobieranie surowych danych pogodowych z zewnętrznego interfejsu Weather REST API, ich strukturyzacja, czyszczenie oraz przygotowanie to zaawansowanej analityki. Cały proces został zaprojektowany w oparciu o popularną w inżynierii danych Architekturę Medalionową (Medallion Architecture), rozdzielającą dane na warstwy o różnym stopniu przetworzenia i wiarygodności.

### 2 Architektura Systemu i Przechowywanie Danych

Proces przetwarzania danych został podzielony na three logiczne etapy, co gwarantuje bezpieczeństwo danych surowych oraz wysoką wydajność zapytań analitycznych na końcu potoku. Głównym magazynem danych (Data Lake) jest usługa Amazon S3.

Wydzielono następujące warstwy (foldery w buckecie S3 `weather-lakehouse-aseied-198344`):

- **bronze/**: Warstwa surowa. Zawiera dane pobrane bezpośrednio z API w natywnym formacie JSON. Stanowi niezmiennie archiwum (Single Source of Truth).
- **silver/**: Warstwa oczyszczona. Przechowuje zwalidowane, odfiltrowane i zdeduplikowane dane w wysoce wydajnym, kolumnowym formacie Parquet.
- **gold/**: Warstwa analityczna. Przechowuje zagregowane podsumowania dzienne (Parquet), zoptymalizowane pod kątem raportowania w systemach BI oraz odpytywania przez silnik Amazon Athena.
- **athena-results/**: Folder techniczny, w którym Amazon Athena automatycznie składa wyniki wykonywanych zapytań SQL.



Rysunek 1: Struktura katalogów w usłudze Amazon S3 implementująca Architekturę Medalionową.

### 3 Struktura Skryptów i Modułowość Kodu

Aplikacja została podzielona na wyspecjalizowane, niezależne skrypty uruchamiane sekwencyjnie. Taka struktura ułatwia utrzymanie i ewentualną rozbudowę potoku w przyszłości:

1. **ingestion.py**: Moduł odpowiedzialny za komunikację z zewnętrznym Weather API. Autoryzuje się tokenem, pobiera najnowsze pomiary (batch) dla zdefiniowanych stacji i zapisuje je w postaci plików JSON w warstwie Bronze, oznaczając pliki sygnaturą czasową.
2. **transformation.py**: Warstwa transformacji (ETL). Wczytuje pliki JSON z warstwy Bronze, formatuje daty (timestamp), usuwa ewentualne duplikaty logów z API i aplikuje reguły walidacyjne (np. odrzucanie nierealnych wartości temperatury czy wilgotności). Wynik zapisuje w warstwie Silver za pomocą biblioteki PyArrow.
3. **analytics.py**: Proces generowania warstwy Gold metodą lokalną. Grupuje dane z warstwy Silver (według stacji i dnia pomiaru), a następnie oblicza agregacje analityczne, generując plik podsumowujący bezpośrednio do S3.
4. Skrypty SQL (**create\_gold\_tables.sql**, **analytical\_queries.sql**): Skrypty DDL i DML przeznaczone dla silnika bezserwerowego Amazon Athena. Służą do tworzenia zewnętrznych tabel na danych z S3 oraz wykonywania zapytań raportujących (np. detekcja anomalii).

### 4 Przetwarzanie Danych i Transformacje (ETL)

Dane w postaci surowej z API mogą zawierać błędy pomiarowe, powtórzenia lub puste wartości. W module **transformation.py** zastosowano szereg mechanizmów czyszczących:

- **Deduplikacja**: Wykorzystano metodę usuwania duplikatów ze szczególnym uwzględnieniem kolumn **station\_id** oraz **timestamp**.

- **Filtrowanie fizyczne (Walidacja):** Skrypt zaczytuje plik konfiguracyjny (JSON) zawierający `VALIDATION_RULES`. Dzięki temu dynamicznie odrzucane są rekordy, w których np. temperatura wykracza poza realne ramy klimatyczne (odrzucając tzw. outliers).
- **Konwersja formatów:** W tym kroku następuje przejście z mało wydajnego analitycznego formatu JSON (Bronze) na skompresowany, kolumnowy format Apache Parquet (Silver), co drastycznie redukuje koszty skanowania danych w chmurze i czas wykonywania zapytań.

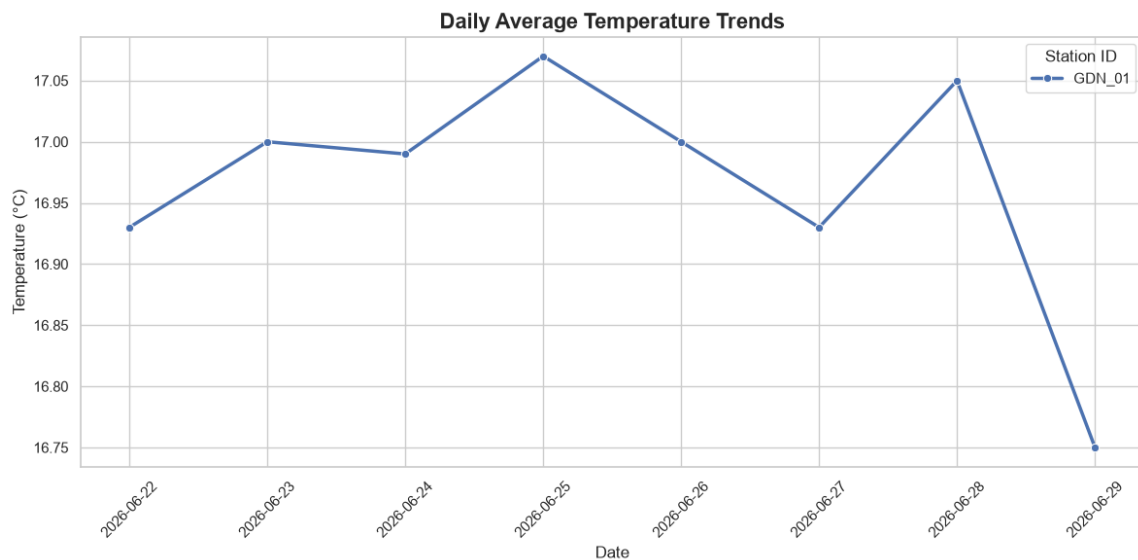
Dla warstwy Gold przeliczane są agregacje dzienne (dzienna średnia, minimalna i maksymalna temperatura, średnia wilgotność oraz sumaryczna ilość opadów z dokładnością do dwóch miejsc po przecinku).

## 5 Metodologia Analityczna i Raportowanie

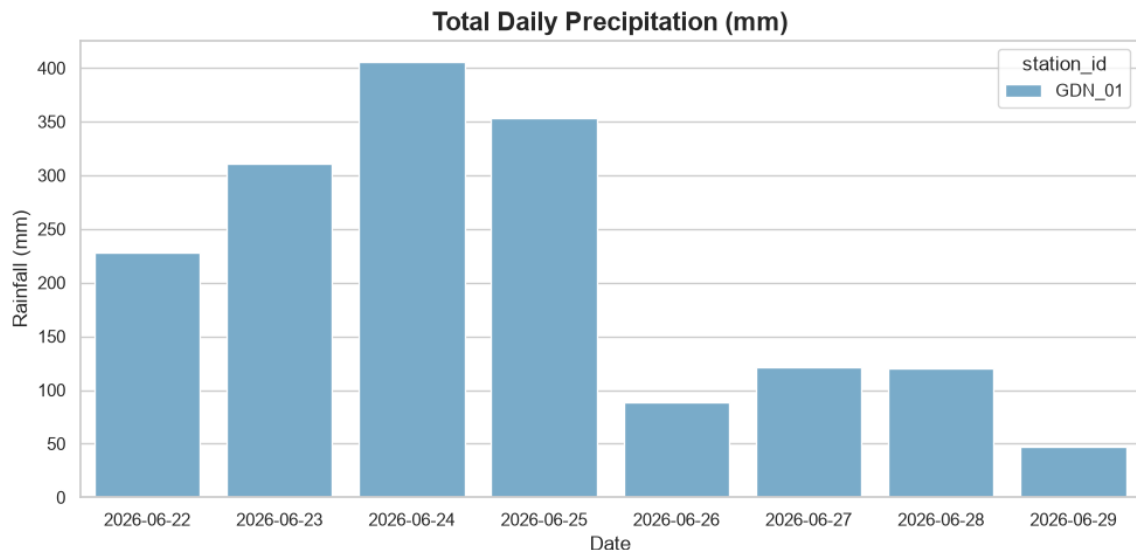
Do ostatecznej weryfikacji i analizy danych wykorzystano AWS Athena. Za pomocą polecenia `CREATE TABLE ... WITH (format = 'PARQUET')` (CTAS) zbudowano analityczną tabelę zewnętrzną `daily_weather_summary`.

Na podstawie wygenerowanych tabel podsumowujących, przygotowano wykresy prezentujące kluczowe metryki pogodowe dla stacji referencyjnej `GDN_01` na przestrzeni badanych dni.

Rysunek 2 prezentuje dzienny trend średniej temperatury, co pozwala na szybką identyfikację zmian termicznych. Z kolei Rysunek 3 obrazuje całkowitą sumę opadów dla poszczególnych dób, co jest weryfikacją poprawności działania reguły agregującej dla opadów.



Rysunek 2: Dzienny trend średniej temperatury (°C) dla stacji `GDN_01` na podstawie danych z warstwy Gold.



Rysunek 3: Całkowita dzienna suma opadów (mm) dla stacji GDN\_01.

## 6 Założenia Projektu

W trakcie budowy architektury Lakehouse przyjęto następujące założenia systemowe:

- **Schema-on-read:** Struktura tabel (schemat) nie jest wymuszana przy zapisie do S3 (z wyjątkiem zachowania formatu Parquet). Schemat jest nakładany dopiero podczas odczytu danych w usłudze Amazon Athena.
- **Rozdzielczość analityczna:** Uznano, że dla celów końcowego raportowania warstwa Gold powinna przechowywać dane zgrubne — o ziarnistości dziennej (dziennie agregacje ze wszystkich pomiarów dla danej stacji).
- **Brak nadpisywania danych (przyrostowość) warstwy Bronze:** Pobieranie danych nigdy nie nadpisuje starych plików. Każde wywołanie API generuje nowy plik JSON z precyzyjnym timestampem w nazwie.

## 7 Ograniczenia Rozwiązania

- **Brak automatycznego rozpoznawania schematów (Crawlers):** W obecnej, podstawowej konfiguracji nowe partycje nie są automatycznie dodawane do AWS Glue Data Catalog. Wymaga to okresowego, ręcznego działania lub rekonfiguracji tabeli w Athenie.